

FSB Executive Intelligence

Documentação Técnica de Arquitetura
Plataforma de inteligência sobre executivos brasileiros

Versão	0.1 (MVP)
Data	28/05/2026
Autoria	Erik Rolin — FGV ECMI Comunicação
Cliente	FSB Holding
Classificação	Documentação técnica — uso interno

1. Sumário executivo

Este documento descreve a arquitetura técnica do MVP **FSB Executive Intelligence**, plataforma de inteligência sobre executivos brasileiros desenvolvida para a FSB Holding. O sistema permite que analistas pesquisem nomes de figuras corporativas relevantes — CEOs, presidentes, board members — e recebam dossiês estruturados consolidando informações públicas: trajetória profissional, presença na imprensa, aparições em eventos, conexões com outros executivos e síntese gerada por modelos de linguagem.

A arquitetura é desenhada em torno de três decisões centrais: **agregação de múltiplas fontes públicas oficiais** em vez de scraping bruto, **uso de LLM (Claude Sonnet) para extração estruturada e síntese** de inteligência narrativa, e construção de **grafo de conexões por co-ocorrência inferida** em mídia, eventos e cargos compartilhados.

2. Visão geral da arquitetura

O backend é estruturado em cinco camadas com responsabilidades isoladas. Cada camada tem ritmo de execução e modelo de falha próprios — separá-las facilita testar, trocar ferramentas e escalar peças individuais sem reescrita.

Coleta → Processamento LLM → Armazenamento → API → Frontend

2.1 Princípios arquiteturais

- **Separação clara entre coleta e apresentação.** O scraping é lento e instável; a API precisa ser rápida. Separar evita que falhas na coleta afetem a experiência do analista.
- **Agregação por API antes de scraping direto.** Cada fonte é tratada como um coletor independente. Fontes oficiais e gerenciadas (Apify, Crunchbase, SerpAPI) são preferidas a scraping bruto do LinkedIn por motivos legais, éticos e operacionais.
- **Processamento por LLM no caminho crítico.** Texto bruto não vira valor por si só. O LLM converte fragmentos em entidades estruturadas e em narrativa sintetizada, que é o entregável real para a equipe de comunicação.
- **Grafo de conexões inferido, não scrapeado.** Relações entre executivos são deduzidas de co-ocorrência em artigos, eventos e cargos compartilhados. Mais defensável legalmente e mais analítico em conteúdo.
- **Cache agressivo e jobs assíncronos.** Buscas duplicadas em janela curta retornam do cache. Coletas novas rodam em background com status acompanhável.

3. Stack técnica

A pilha é predominantemente Python, com foco em ferramentas amplamente adotadas e bem documentadas. O critério de escolha priorizou maturidade, qualidade de documentação em português e baixa fricção operacional para o autor manter sozinho.

Camada	Ferramenta	Função
Linguagem	Python 3.11+	Base do backend

Camada	Ferramenta	Função
API	FastAPI + Pydantic v2	Servidor HTTP, validação de schemas
Banco de dados	PostgreSQL 16 + pgvector	Relacional + busca semântica futura
ORM	SQLAlchemy 2	Acesso ao banco
Migrations	Alembic	Versionamento do schema
Cache / Fila	Redis + RQ	Cache de buscas e fila de jobs
Scraping	Playwright (reserva)	Casos não cobertos por API
LinkedIn	Apify SDK	Coleta gerenciada de perfis
LLM	Anthropic SDK (Claude)	Extração e síntese
HTTP	httpx	Chamadas a APIs externas
Auth	python-jose (JWT)	Autenticação interna
Logs	loguru	Observabilidade simples
Testes	pytest	Suite de testes
Qualidade	ruff + black	Linting e formatação

4. Camada de coleta

A coleta é montada como uma **constelação de fontes**. O LinkedIn é apenas uma delas, o que reduz dependência e aumenta robustez. Cada coletor vive em um módulo independente sob `app/services/collectors/` e expõe uma função pura que recebe parâmetros e devolve um dicionário cru, sem efeitos colaterais em banco.

Fonte	Tipo	O que entrega
Apify (LinkedIn)	Terceirizada	Perfil, experiências, posts públicos
Crunchbase API	Oficial	Carreira corporativa, board, M&A
SerpAPI	Oficial	Imprensa brasileira via Google programático
GDELT	Pública gratuita	Eventos e menções globais
B3 / CVM	Pública gratuita	Comunicados, transações de insiders
Receita / CNPJ	Pública gratuita	Quadros societários

4.1 Decisões sobre LinkedIn

Scraping direto do LinkedIn foi descartado como caminho primário. A justificativa é tripla: **legal** (vai contra os termos de serviço da plataforma), **técnica** (detecção agressiva exige proxy rotativo e quebra com frequência) e **reputacional** (a FSB é uma empresa de comunicação séria; depender de dados raspados em produção é risco assimétrico). Optou-se pelo Apify, que gerencia a operação e absorve parte do risco técnico em troca de uma taxa baixa por perfil.

5. Camada de processamento LLM

O processamento é onde os fragmentos de texto coletados viram inteligência estruturada. Três tipos de chamada compõem a camada, cada um com prompt especializado.

5.1 Extrator estruturado

Recebe um texto bruto (post, artigo, bio) e devolve um JSON validado por Pydantic contendo *eventos, empresas mencionadas, pessoas mencionadas, valores monetários, datas, sentimento e temas*. Usa `tool_use` da API para forçar o schema — não confia em 'devolva JSON'.

5.2 Sintetizador de perfil

Após todas as extrações, uma chamada final recebe o conjunto consolidado e produz um briefing executivo em três parágrafos: posicionamento público, temas defendidos e tom na mídia, e momentos-chave dos últimos 12 meses. Esse é o entregável de leitura humana consumido pela equipe da FSB.

5.3 Inferidor de relações

Recebe pares de pessoas e o contexto onde apareceram juntas. Devolve *{tipo da relação, força, evidência}*. Ex: 'ambos no board do mesmo M&A; em 2024' → relação forte com evidência citada. Esse passo alimenta o grafo de conexões.

6. Modelo de dados

O banco é relacional. Sete entidades principais cobrem o domínio. A modelagem permite navegação tanto biográfica (uma pessoa e seus cargos) quanto relacional (o grafo de conexões entre pessoas).

Entidade	Função
Pessoa	Executivo individual: nome, cargo atual, bio, foto, briefing
Empresa	Companhia: nome, setor, CNPJ, ticker B3
Cargo	Ponte Pessoa↔Empresa com função, datas e flag 'é atual'
Relacao	Aresta Pessoa↔Pessoa do grafo: tipo, peso, lista de evidências
Evento	Encontro público (Davos, Lide, conferências) com participantes
Mencao	Aparição em mídia: fonte, URL, texto, sentimento, temas
JobColeta	Status de coleta assíncrona (queued, running, done, failed)

7. Camada de API

A API é em FastAPI. Quatro routers cobrem o fluxo principal. Jobs assíncronos garantem que o cliente não fique pendurado durante coletas que levam dezenas de segundos.

Endpoint	Método	Função
/busca	POST	Cria ou recupera perfil. Retorna job_id ou cache hit.
/perfil/{id}	GET	Dossiê completo: cargos, menções, briefing.
/grafo/{id}	GET	Grafo de conexões em formato pronto para Cytoscape.
/job/{id}	GET	Status de coleta assíncrona.

8. Fluxo end-to-end de uma busca

A sequência abaixo descreve o que acontece desde a digitação do nome até a renderização do dossiê pelo frontend.

1. Analista da FSB envia POST /busca com o nome do executivo.
2. API normaliza o nome em slug canônico e verifica cache.
3. Se cache fresco existe (< 7 dias), retorna direto. Caso contrário, cria JobColeta e enfileira no Redis.
4. Worker RQ dispara coletores em paralelo: Apify, Crunchbase, SerpAPI, GDELT, B3, Receita.
5. Cada resposta bruta passa pelo extrator LLM, que devolve entidades estruturadas.
6. Persistência: Pessoa, Cargo, Empresa, Evento e Mencao são populados.
7. Pessoas co-mencionadas geram ou reforçam arestas em Relacao no grafo.
8. Sintetizador LLM gera o briefing executivo em português a partir do conjunto consolidado.
9. Job marcado como 'done'. Frontend faz polling em /job e busca /perfil e /grafo.

9. Estrutura de diretórios

A árvore abaixo reflete o estado atual do repositório. Cada subdiretório de *services/* isola uma responsabilidade — coletores externos, camada LLM, construção e consulta de grafo.

```
fsb-executive-intelligence/  
■■■ README.md  
■■■ requirements.txt  
■■■ .env.example  
■■■ .gitignore  
■■■ app/  
■   ■■■ main.py  
■   ■■■ core/      (config, db, security)  
■   ■■■ api/       (busca, perfil, grafo, job)  
■   ■■■ models/    (7 entidades SQLAlchemy)  
■   ■■■ schemas/   (Pydantic request/response)  
■   ■■■ services/  
■   ■   ■■■ collectors/ (apify, crunchbase, serpapi, gdelt, b3, receita)  
■   ■   ■■■ llm/      (extrator, sintetizador, inferidor_relacoes)
```

```
■ ■ ■■■ graph/      (construtor, queries)
■ ■ ■■■ cache.py
■ ■■■ workers/      (busca_worker)
■■■ tests/
```

10. Decisões arquiteturais registradas

As decisões abaixo foram tomadas conscientemente no início do projeto. Cada uma carrega trade-offs que podem ser revisitados se o escopo mudar.

- **Sem scraping direto agressivo do LinkedIn.** Risco legal e reputacional para a FSB. Apify gerencia risco técnico em troca de taxa baixa.
- **LLM desde o MVP.** Custo absorvível; qualidade dos relatórios justifica e diferencia o produto.
- **Grafo por co-ocorrência, não por scraping de rede social.** Mais ético, mais defensável e tecnicamente mais analítico.
- **Postgres em vez de Neo4j para o grafo.** CTEs recursivas atendem o volume de MVP sem operar um segundo banco.
- **RQ em vez de Celery para jobs.** Simplicidade vence em fase de MVP. Migração futura é viável.
- **Local-first para desenvolvimento.** Dockerização será adicionada quando o escopo for validado com a FSB.
- **pgvector habilitado desde o início.** Custa pouco adicionar e abre caminho para busca semântica sobre perfis e posts.

11. Segurança e gestão de credenciais

Todas as credenciais de APIs externas e segredos de autenticação são carregados de variáveis de ambiente via *pydantic-settings*. Um arquivo *.env.example* documenta os nomes das variáveis necessárias, mas o arquivo real *.env* não é versionado — está listado no *.gitignore* do projeto.

Autenticação interna usa JWT com segredo configurável. Para produção, recomenda-se rotação periódica e armazenamento dos segredos em um gerenciador dedicado (AWS Secrets Manager, GCP Secret Manager ou equivalente), substituindo o *.env* local.

12. Próximos passos

- Configuração das contas nos serviços externos e ativação das chaves.
- Implementação iterativa por camada, começando pelos coletores prioritários.
- Inicialização do Alembic e criação das migrations iniciais.
- Implementação efetiva das chamadas LLM com prompts validados.
- Construção do frontend (definição de stack em fase posterior).
- Validação de cenários reais de uso com a equipe da FSB.
- Dockerização do ambiente para entrega.

Documento gerado automaticamente a partir da arquitetura definida em conjunto. Sujeito a revisões conforme validação com a FSB Holding.